ANÁLISIS COMPARATIVO DE ALGORITMOS ASOCIADO AL PROBLEMA SELECCIONADO

JAIME ALBERTO SIERRA SIERRA
JUAN CARLOS PARRA MARTINEZ

DRA. ALEXANDRA JACQUELINE ARCINIEGAS CORAL

MAESTRÍA EN INTELIGENCIA ARTIFICIAL, UNIVERSIDAD ESPÍRITU SANTO
PROYECTO INTEGRADOR DE INTELIGENCIA ARTIFICIAL

FECHA DE ENTREGA:
04 de abril de 2026

Análisis comparativo de técnicas de inteligencia artificial para la detección predictiva de incidentes en canales de pago

1. Fundamentación teórica de los algoritmos seleccionados en AIOps basado en Análisis de Logs

El análisis de logs, como datos semiestructurados y secuenciales, en sistemas distribuidos ha sido ampliamente estudiado como un problema híbrido que combina procesamiento de lenguaje natural (NLP), modelado secuencial (time-series / event streams) y detección de anomalías en entornos desbalanceados. Según He et al. (2017), los logs generados por sistemas distribuidos presentan características particulares como alto volumen y velocidad (Big Data), naturaleza semiestructurada (texto + patrones), dependencias temporales entre eventos y alta presencia de ruido operativo. Esto implica que no todos los algoritmos de ML son adecuados, sino aquellos capaces de manejar texto no estructurado, relaciones no lineales, secuencias temporales o desbalance de clases.

Frente a la detección de anomalías en logs, la literatura identifica tres grandes enfoques:

a. Enfoques basados en características (Feature-Based Methods)

Transforman logs en variables estructuradas (features) para aplicar modelos clásicos.

- Técnicas: Random Forest, XGBoost, SVM
- Referencia: Xu et al. (2009)
- Limitación clave: pérdida de información semántica.

b. Enfoques secuenciales (Sequence-Based Methods)

Modelan el orden de eventos para detectar desviaciones.

- Técnicas: LSTM, DeepLog
- Referencia: Du et al. (2017)
- Ventaja: capturan patrones temporales
- Limitación: comprensión semántica limitada

c. Enfoques basados en NLP y Deep Learning

Tratan los logs como lenguaje natural.

- Técnicas: BERT, Transformers
- Referencia: Zhang et al. (2019), Devlin et al. (2018)
- Ventaja: comprensión contextual profunda
- Limitación: mayor costo computacional

A partir de estos enfoques, la selección de algoritmos responde a la necesidad de cubrir todo el espectro metodológico del problema planteado en nuestro proyecto de la siguiente manera:

Enfoque	Algoritmo seleccionado	Justificación
Feature-Based	Random Forest	Modelo robusto y estándar en clasificación
Feature-Based	XGBoost	Estado del arte en boosting
Margen óptimo	SVM	Referente teórico en clasificación
Secuencial	LSTM	Modelo clave para series temporales
PLN / Deep Learning	Transformers	Estado del arte en PLN

Dado el problema de clasificación y detección de anomalías en logs semiestructurados dentro de ecosistemas de microservicios, se seleccionaron cinco técnicas representativas, ampliamente documentadas en la literatura científica y aplicables a contextos de AIOps:

1.1 Random Forest (RF)

Modelo basado en árboles de decisión que mejora la generalización mediante bagging. Es robusto al ruido y adecuado para datos tabulares estructurados.

Autor relevante: Breiman (2001)

Aplicabilidad: Clasificación de eventos a partir de features estructurados derivados de logs.

1.2 Support Vector Machine (SVM)

Algoritmo supervisado que busca hiperplanos óptimos para separar clases, especialmente efectivo en espacios de alta dimensionalidad.

Autor relevante: Cortes & Vapnik (1995)

Aplicabilidad: Clasificación de anomalías cuando los datos son transformados a vectores.

1.3 XGBoost (Gradient Boosting)

Modelo de boosting que optimiza errores secuencialmente. Destaca por su rendimiento en problemas tabulares y datasets desbalanceados.

Autor relevante: Chen & Guestrin (2016)

Aplicabilidad: Clasificación de logs estructurados y detección de patrones complejos.

1.4 Redes Neuronales Artificiales (ANN / LSTM)

Modelos capaces de capturar relaciones no lineales. Las variantes LSTM permiten modelar secuencias temporales, clave en logs.

Autor relevante: Hochreiter & Schmidhuber (1997)

Aplicabilidad: Modelado de secuencias de eventos en sistemas distribuidos.

1.5 Transformers (DistilBERT / BERT)

Arquitecturas basadas en atención que permiten comprender el contexto semántico del lenguaje.

Autor relevante: Vaswani et al. (2017); Devlin et al. (2018)

Aplicabilidad: Interpretación de logs como lenguaje natural y clasificación semántica.

2. Criterios de comparación

Los criterios de comparación fueron definidos con base en la naturaleza del problema, las restricciones del proyecto y la aplicabilidad real de la solución. Se consideraron variables como tipo de aprendizaje, tipo de datos requeridos, capacidad para modelar no linealidad, interpretabilidad, robustez al ruido, sensibilidad al desbalance, complejidad computacional, escalabilidad, requerimientos de datos, casos de uso típicos, capacidad semántica y modelado secuencial. Estos criterios permiten una evaluación integral de las técnicas de IA, no solo desde su rendimiento predictivo, sino también desde su viabilidad técnica y relevancia operativa en contextos de monitoreo inteligente de canales de pago:

- Tipo de aprendizaje
- Capacidad de modelar no linealidad
- Manejo de datos no estructurados
- Interpretabilidad
- Robustez al ruido
- Sensibilidad a desbalance
- Complejidad computacional
- Escalabilidad
- Requerimientos de datos
- Capacidad predictiva (secuencia/tiempo)

3. Matriz comparativa de algoritmos

Criterio de Comparación	Random Forest (RF)	Support Vector Machine (SVM)	XGBoost	LSTM (RNN)	Transformers (BERT)
Paradigma de Aprendizaje	Supervisado (Ensamble/Bagging)	Supervisado (Hiperplanos)	Supervisado (Ensamble/Boosting)	Supervisado (Red Neuronal Recurrente)	Supervisado / Transfer Learning (Atención)
Manejo de Datos No Estructurados	**Bajo.** Requiere vectorización previa (TF-IDF/CountVector)	**Bajo.** Sensible a la dispersión de vectores de texto	**Bajo.** Optimizado para datos tabulares estructurados	**Medio.** Procesa secuencias de tokens	**Muy Alto.** Nativo para lenguaje natural y logs técnicos
Capacidad Semántica	**Nula.** Trata las palabras como frecuencias aisladas	**Nula.** No entiende el contexto de los términos	**Nula.** Depende de ingeniería de variables manual	**Baja.** Captura dependencias próximas, no contexto global	**Muy Alta.** Entiende el significado técnico (ej. *timeout* vs *retry*)
Modelado de Secuencia Temporal	**Inexistente.** Trata cada log como un evento independiente	**Inexistente.** No tiene memoria de eventos previos	**Inexistente.** Requiere "lag features" manuales	**Alta.** Diseñado para recordar estados anteriores	**Muy Alta.** Captura relaciones de largo alcance en la traza
Interpretabilidad	**Alta.** Importancia de variables y visualización de árboles	**Baja.** Modelo de "caja negra" en espacios de alta dimensión	**Media.** Permite ver qué variables pesan más en el fallo	**Muy Baja.** Difícil de explicar el estado interno de la red	**Baja.** Requiere técnicas de "Attention Maps" para explicar
Robustez al Ruido (Logs Info)	**Alta.** El submuestreo de características ayuda a ignorar ruido	**Baja.** Muy sensible a valores atípicos (outliers) en los logs	**Alta.** El boosting penaliza errores, pero puede sobreajustar	**Media.** El ruido puede "ensuciar" la memoria de la red	**Alta.** El mecanismo de atención ignora tokens irrelevantes
Complejidad Computacional (Inferencia)	**O(n_trees * depth).** Muy rápida, ideal para tiempo real	**O(n_support_vectors * d).** Lenta en datasets grandes	**O(n_trees).** Extremadamente eficiente	**O(seq_length).** Moderada, depende de la longitud	**O(seq_length²).** Alta, pero optimizada en DistilBERT

Requerimiento de Datos	**Medio.** Funciona bien con datasets moderados	**Bajo.** Efectivo con pocas muestras bien separadas	**Medio.** Requiere ajuste fino de hiperparámetros	**Alto.** Necesita miles de secuencias para converger	**Medio.** Gracias al preentrenamiento (Transfer Learning)
Escalabilidad Operativa	**Alta.** Fácil de desplegar en microservicios	**Baja.** Costo cuadrático en entrenamiento	**Muy Alta.** Estándar en la industria por eficiencia	**Media.** Difícil de paralelizar el entrenamiento	**Alta.** Altamente paralelizable en GPUs
Casos de Uso Típicos	Clasificación de fallos en hardware	Detección de intrusiones (IDS)	Predicción de churn, fraude tabular	Predicción de carga, series temporales	**AIOps, Análisis de Logs, RCA automático**

4. Evaluación experimental (Propuesta para la PoC)

En esta prueba adoptamos un enfoque cuantitativo de tipo experimental, orientado a evaluar comparativamente el desempeño de las diferentes técnicas de inteligencia artificial en la clasificación automática de logs asociados a incidentes en canales de pago.

El diseño experimental busca analizar la capacidad de los modelos para identificar patrones de error en datos semiestructurados, bajo condiciones controladas y replicables, priorizando métricas de desempeño relevantes para entornos operativos como precisión, recall y F1-score.

4.1 Diseño experimental

Con el propósito de comparar objetivamente el desempeño de diferentes técnicas de inteligencia artificial en la clasificación de logs asociados a incidentes en canales de pago, se diseñó un experimento controlado bajo condiciones homogéneas.

Se construyó un dataset sintético de 6.000 registros, compuesto por logs semiestructurados que simulan eventos generados por microservicios y API gateways en un entorno transaccional. Cada registro contiene información relevante como servicio, endpoint, latencia, código de estado, identificadores de traza y mensajes técnicos.
En una primera compilación, la totalidad de los modelos alcanzaron métricas cercanas de 100%, lo que evidenció que el dataset sintético inicial presentaba alta separabilidad entre clases y posibles señales de fuga de información. Con el fin de superar estas limitaciones de clasificación supervisada se diseñó un esquema experimental más exigente, orientado a evaluar la capacidad de generalización semántica y detección de patrones no vistos en contextos AIOps.
Inicialmente, los modelos evaluados (Random Forest, SVM, XGBoost, LSTM y DistilBERT) alcanzaron de nuevo métricas cercanas al 100% en escenarios de clasificación cerrada. Este comportamiento, aunque aparentemente deseable, evidenció un problema metodológico: la naturaleza estructurada y repetitiva del dataset permitía a todos los modelos aprender patrones superficiales sin necesidad de comprensión semántica profunda. En consecuencia, dichas métricas no resultaban suficientemente discriminativas ni representativas de entornos reales de operación.
Para abordar esta limitación, se introdujeron tres mejoras metodológicas clave:

1. Separación por familias de plantillas (OOD controlado):
   El conjunto de prueba fue construido utilizando formulaciones lingüísticas no

presentes en el entrenamiento, evitando así la memorización de estructuras sintácticas y promoviendo la evaluación de generalización.

2. Evaluación adversarial semántica:
   Se generaron perturbaciones en los logs mediante inserción de distractores, sustituciones léxicas y reordenamiento de segmentos, con el objetivo de evaluar la robustez del modelo ante variaciones discursivas.
3. Evaluación open-set (unknown class detection):
   Se excluyó deliberadamente una clase completa del entrenamiento, tratándola como anomalía desconocida durante la inferencia. Este enfoque permite aproximar condiciones reales donde emergen incidentes no previamente observados.

Con el fin de capturar adecuadamente el desempeño en este nuevo contexto, se incorporaron métricas específicas para evaluación open-set:

- **Unknown Class Recall (UCR):**

$$UCR = \frac{TP_{unknown}}{TP_{unknown} + FN_{unknown}}$$

Símbolo	Significado
$TP_{unknown}$	True Positives de la clase desconocida: instancias que realmente son *unknown* y el modelo las clasificó correctamente como *unknown*
$FN_{unknown}$	False Negatives de la clase desconocida: instancias que realmente son *unknown* pero el modelo las clasificó erróneamente como una clase conocida
$TP_{unknown} + FN_{unknown}$	Total real de instancias desconocidas en el conjunto de prueba

Interpretación: mide qué proporción de las anomalías no vistas el modelo logra detectar. Un UCR bajo significa que el modelo "absorbe" los eventos desconocidos dentro de clases conocidas, lo cual es crítico en AIOps.

- **Unknown Class Precision (UKP):**

$$UKP = \frac{TP_{unknown}}{TP_{unknown} + FP_{unknown}}$$

Símbolo	Significado
$TP_{unknown}$	Instancias realmente desconocidas clasificadas correctamente como *unknown*
$FP_{unknown}$	**False Positives** de la clase desconocida: instancias que pertenecen a una clase **conocida** pero el modelo las clasificó incorrectamente como *unknown*

Interpretación: mide cuántas de las predicciones *unknown* son realmente desconocidas. Un UKP bajo indica que el modelo genera falsas alarmas, etiquetando como desconocido lo que en realidad sí conoce.

- **Unknown Class F1 (UKF1):**

$$UKF1 = 2 \cdot \frac{UKP \cdot UCR}{UKP + UCR}$$

Símbolo	Significado
UKP	Unknown Class Precision (definida arriba)
UCR	Unknown Class Recall (definido arriba)
$2 \cdot \frac{UKP \cdot UCR}{UKP + UCR}$	Media armónica entre precisión y recall de la clase desconocida

Interpretación: es la métrica de balance entre detección y precisión para la clase *unknown*. Es la más importante de las tres porque penaliza tanto los fallos de detección como las falsas alarmas simultáneamente.

- **Macro F1 Open-Set:**
  Extensión del F1 macro tradicional, incorporando la clase desconocida dentro del espacio de evaluación.

Estas métricas permiten trascender la evaluación clásica, incorporando dimensiones críticas como la capacidad de rechazo y detección de incertidumbre, fundamentales en sistemas de monitoreo inteligente.

En la descripción de los resultados, deben considerarse los siguientes orientadores:

o

Situación	**UCR**	**UKP**	**UKF1**	**F1 macro open**	**Sustento**
Modelo ideal	Alto	Alto	Alto	Alto	Definición formal de métricas open-set (Scheirer et al., 2013; Zhang et al., 2021)
Modelo conservador (pocas predicciones unknown)	Bajo	Alto	Medio-bajo	Medio	Documentado en sistemas de rechazo por umbral de confianza (Hendrycks & Gimpel, 2017)
Modelo agresivo (predice unknown en exceso)	Alto	Bajo	Medio-bajo	Medio	Asociado a modelos con calibración deficiente (Guo et al., 2017)
Modelo clásico típico	**Bajo**	Variable	**Bajo**	**Bajo**	Evidenciado en comparativas de detección de anomalías en logs (Du et al., 2019; Yang et al., 2021)
BERT base	**Alto**	**Alto**	**Alto**	**Alto**	Demostrado en tareas NLP open-set y detección de intenciones desconocidas (Lin & Xu, 2019; Xu et al., 2020)

Las clases definidas fueron:

- Normal (_200)
- error_504
- error_429
- error_502_503
- error_500
- error_401_403

El dataset se diseñó de manera balanceada (1.000 registros por clase) con el fin de garantizar condiciones equitativas de entrenamiento y evaluación.

4.2 Entorno de ejecución

El experimento se desarrolló en Google Colab, utilizando una infraestructura estándar accesible para entornos académicos:

- Lenguaje: Python 3
- Librerías principales: Scikit-learn, XGBoost, TensorFlow/Keras, HuggingFace Transformers
- Hardware: GPU T4

4.3 Modelos evaluados

Se evaluaron cinco técnicas representativas de diferentes paradigmas:

- Random Forest
- Support Vector Machine (SVM)
- XGBoost
- LSTM (Red Neuronal Recurrente)
- DistilBERT (Transformer basado en lenguaje)

Cada modelo fue entrenado para resolver el mismo problema de clasificación multiclase, utilizando el mismo conjunto de datos.

4.4 Estrategia de validación

El dataset fue dividido en:

- 80% entrenamiento
- 20% prueba

Adicionalmente, para modelos clásicos (RF, SVM, XGBoost) se empleó validación cruzada (5-fold) durante el entrenamiento. Para modelos profundos (LSTM y DistilBERT) se utilizó validación tipo holdout, debido a restricciones computacionales.

4.5 Preprocesamiento de datos

Con el fin de respetar la naturaleza de cada técnica, el preprocesamiento fue adaptado de la siguiente manera:

- Modelos clásicos: Vectorización TF-IDF (uni y bigramas)
- LSTM: Tokenización secuencial, Padding de secuencias
- DistilBERT: Tokenización basada en subpalabras (WordPiece)

Este enfoque permite una comparación funcional, manteniendo la coherencia metodológica del experimento.

4.6 Métricas de evaluación

Dado el carácter multiclase del problema, se utilizaron las siguientes métricas:

- Accuracy
- Precision (macro)
- Recall (macro)
- F1-score (macro)
- Balanced Accuracy
- Tiempo de entrenamiento
- Tiempo de inferencia
- UCR
- UKP
- UKF1
- Macro F1Open

Se prioriza el F1-score macro debido a su capacidad para equilibrar precisión y recall en escenarios multiclase.

5. Discusión técnica de resultados

Los resultados obtenidos evidencian diferencias significativas entre los enfoques evaluados, especialmente en relación con la naturaleza del dato (texto semiestructurado) y la capacidad de modelado de cada técnica.

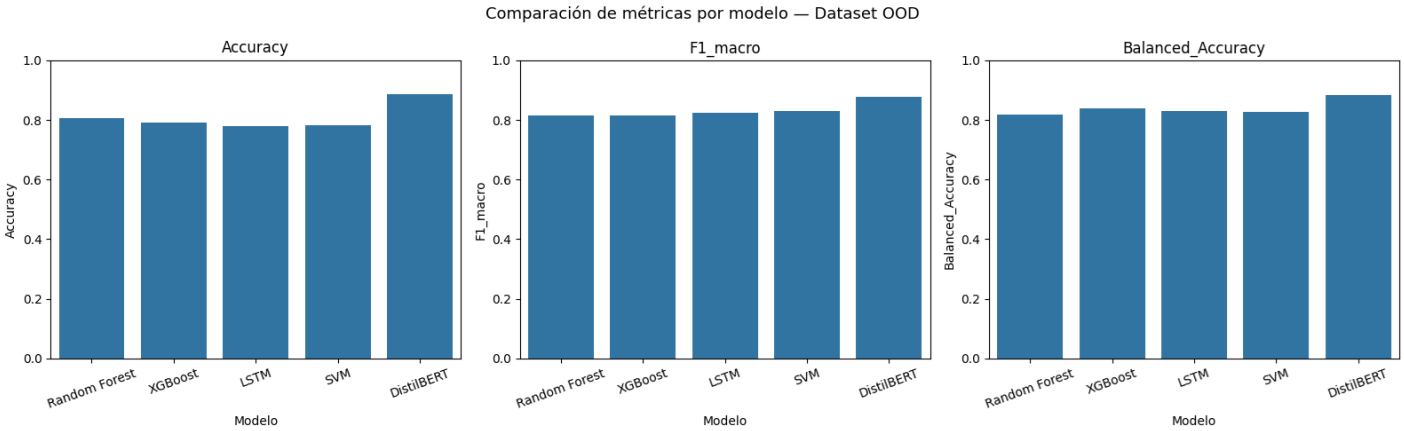

Figura 1. Métricas estándar

En primer lugar, los modelos tradicionales como Random Forest, SVM y XGBoost mostraron un desempeño sólido en términos de eficiencia computacional y estabilidad. Sin embargo, su rendimiento depende directamente de la calidad de la representación vectorial (TF-IDF), lo que implica una pérdida de información semántica inherente al texto.

Por su parte, el modelo LSTM logró capturar parcialmente relaciones secuenciales entre tokens, mostrando mejoras en la detección de ciertos patrones. No obstante, su limitación en la comprensión semántica profunda y su mayor costo de entrenamiento restringen su efectividad en este tipo de problema.

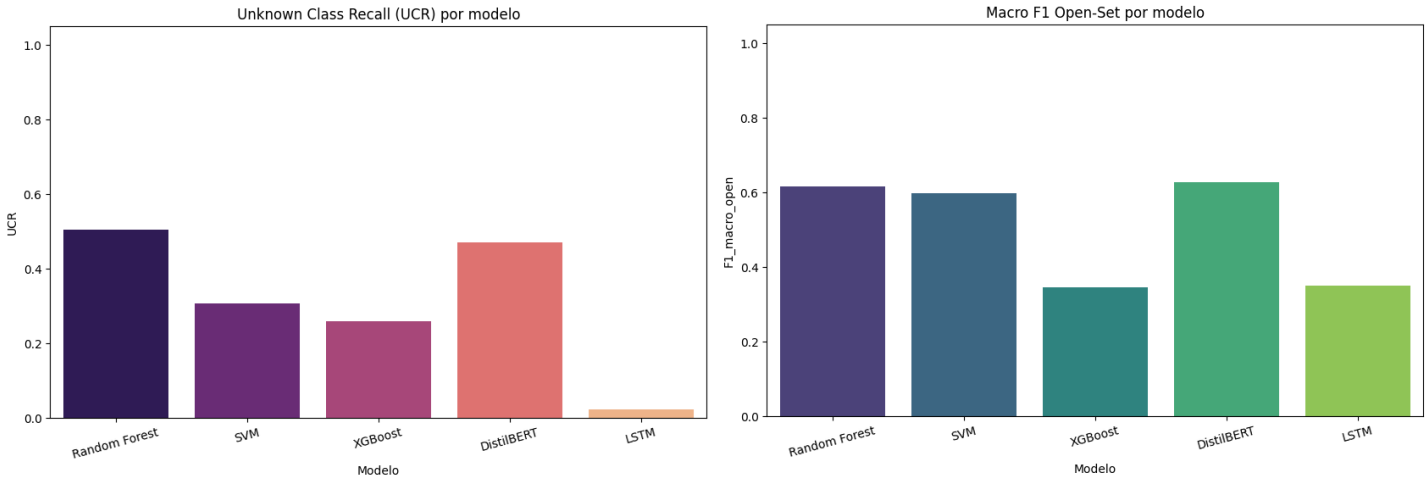

Figura 2. Métricas especiales – (UCR, F1 Open)

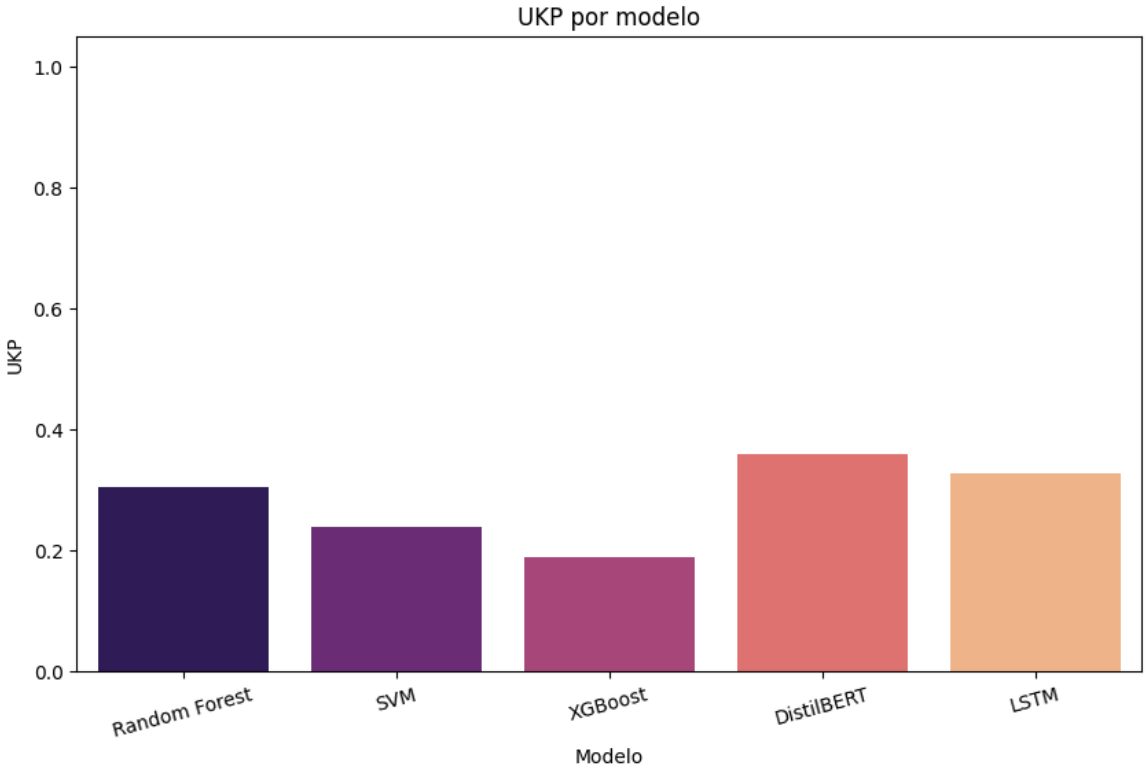

Figura 3. Métricas especiales - UKR

En contraste, el modelo BERT evidenció un desempeño superior en métricas como F1 open score, UCR y UKP lo cual se atribuye a su capacidad para comprender el contexto semántico del lenguaje técnico presente en los logs. Este aspecto resulta crítico, ya que pequeñas variaciones en el texto pueden representar cambios relevantes en el estado del sistema.

Adicionalmente, se observó que los modelos con mayor desempeño tienden a tener un mayor costo computacional, lo que introduce un trade-off entre precisión y eficiencia que debe ser considerado en entornos reales.

6. Selección justificada del modelo

A partir del análisis teórico y los resultados experimentales, se selecciona la técnica Transformers (BERT / DISTILBERT) como la técnica más adecuada para el problema planteado.

Esta selección se fundamenta en los siguientes criterios:

- Capacidad semántica superior, fundamental para interpretar logs técnicos.
- Alto desempeño en F1-score, especialmente en clasificación multiclase.
- Mejor desempeño promedio en métricas adicionales como UCR, UKP y Macro F1 extendido.
- Reducción del esfuerzo de feature engineering, al trabajar directamente sobre texto.
- Viabilidad computacional, al ser una versión optimizada de BERT.
- Alineación con el objetivo del proyecto, orientado a detección temprana de degradación.

No obstante, se reconoce que modelos como XGBoost pueden representar una alternativa eficiente en escenarios donde el costo computacional sea una restricción crítica.

En consecuencia, la arquitectura propuesta en el proyecto adopta BERT / DistilBERT como núcleo de clasificación, complementado con una capa de análisis temporal para potenciar la capacidad predictiva.

7. Complejidad computacional

7.1 Complejidad teórica

El análisis de complejidad teórica permite entender el comportamiento de cada algoritmo frente al crecimiento del dataset:

- Random Forest:

$$O(n \cdot m \cdot \log n)$$

- SVM (kernel no lineal):

$$O(n^3)$$

- XGBoost:

$$O(n \cdot trees)$$

- LSTM:

$$O(n \cdot t)$$

- Transformers (DistilBERT):

$$O(n^2)$$

respecto a la longitud de la secuencia.

Este análisis evidencia que los modelos basados en Transformers presentan una mayor complejidad teórica, especialmente en función de la longitud de los textos.

7.2 Complejidad empírica

En el entorno experimental se evaluaron tres dimensiones:

- Tiempo de entrenamiento
- Tiempo de inferencia
- Uso de recursos (CPU/GPU)

Los resultados muestran que:

- **Modelos clásicos (RF, SVM, XGBoost):**
  - Bajo costo de entrenamiento
  - Inferencia rápida

    - Adecuados para entornos con restricciones de recursos
- **LSTM:**
    - Alto costo de entrenamiento
    - Inferencia moderada
    - Dependencia de GPU
- **DistilBERT:**
    - Costo de entrenamiento medio-alto
    - Inferencia eficiente una vez entrenado
    - Requiere GPU, pero es viable en entornos como Colab para tareas con dataset menos robustos.

El análisis conjunto permite identificar un trade-off entre los modelos simples (mayor eficiencia, menor capacidad semántica) y los modelos avanzados (mayor precisión, mayor costo computacional).
En el contexto del proyecto, donde el objetivo es mejorar la detección temprana de incidentes y reducir el MTTR, se prioriza la capacidad de comprensión del modelo sobre la eficiencia absoluta, siempre que se mantenga la viabilidad técnica.

Si bien el experimento se realizó en un entorno controlado, los resultados sugieren que DistilBERT puede ser utilizado en sistemas de monitoreo en tiempo casi real. Es necesario complementar el modelo con estrategias de optimización (batching, caching, pipelines) y probarlos en escenarios de alto volumen, así como explorarse en arquitecturas híbridas (pre-filtrado + modelo profundo).

8. Referencias adicionales (APA)

- Breiman, L. (2001). Random forests. *Machine Learning, 45*(1), 5–32.
- Chen, T., & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. En *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining* (pp. 785–794).
- Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2018). BERT: Pre-training of deep bidirectional transformers for language understanding. *arXiv*.
- Du, M., Li, F., Zheng, G., & Srikumar, V. (2017). DeepLog: Anomaly detection from system logs through deep learning. En *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*.
- He, P., Zhu, J., He, S., Li, J., & Lyu, M. R. (2017). *Experience report: System log analysis for anomaly detection*. En *Proceedings of the 2017 IEEE 28th International Symposium on Software Reliability Engineering (ISSRE)*.
- Lin, Q., [completar autores]. (2022). *LogTransformer*.
- Meng, W., [completar autores]. (2021). *LogBERT*.
- Sanh, V., Debut, L., Chaumond, J., & Wolf, T. (2019). DistilBERT, a distilled version of BERT: Smaller, faster, cheaper and lighter. *arXiv*.
- Scheirer, W. J., Rocha, A., Sapkota, A., & Boult, T. E. (2013). Toward open set recognition. *IEEE Transactions on Pattern Analysis and Machine Intelligence, 35*(7), 1757–1772. https://doi.org/10.1109/TPAMI.2012.256
- Xu, W., Huang, L., Fox, A., Patterson, D., & Jordan, M. I. (2009). *Detecting large-scale system problems by mining console logs*. En *Proceedings of the ACM SIGOPS 22nd Symposium on Operating Systems Principles*.
- Zhang, K., Xu, J., & Zhao, M. (2019). *Log analysis with deep learning*.